In [1]:
```python
import pandas as pd

def build_combined_dataset():
    # Load source datasets — ensure CSVs are in the same working directory
    orders = pd.read_csv('olist_orders_dataset.csv')
    reviews = pd.read_csv('olist_order_reviews_dataset.csv')
    customers = pd.read_csv('olist_customers_dataset.csv')

    print(f"Orders loaded: {len(orders)}")
    print(f"Customers loaded: {len(customers)}")
    print(f"Reviews loaded: {len(reviews)}")

    # Convert timestamps and de-duplicate reviews by keeping the latest response per order
    reviews['review_answer_timestamp'] = pd.to_datetime(reviews['review_answer_timestamp'])
    reviews_clean = (
        reviews
        .sort_values('review_answer_timestamp', ascending=False)
        .drop_duplicates(subset='order_id', keep='first')
    )
    print(f"Reviews after deduplication: {len(reviews_clean)}")

    # Join orders with reviews, then with customers
    orders_with_reviews = orders.merge(reviews_clean, how='left', on='order_id')
    combined = orders_with_reviews.merge(customers, how='left', on='customer_id')

    # Validation check
    print("-" * 40)
    print(f"Combined dataset rows: {len(combined)}")
    if len(combined) == len(orders):
        print("CHECK PASSED: Row count matches orders.")
    else:
        print(f"WARNING: Row count differs by {len(combined) - len(orders)} rows.")

    combined.to_csv('combined_dataset.csv', index=False)
    print("Saved as 'combined_dataset.csv'")

if __name__ == "__main__":
    build_combined_dataset()
```

```
Orders loaded: 99441
Customers loaded: 99441
Reviews loaded: 99224
Reviews after deduplication: 98673
----------------------------------------
Combined dataset rows: 99441
CHECK PASSED: Row count matches orders.
Saved as 'combined_dataset.csv'
```

In [2]:
```
# Load the combined dataset and preview
df = pd.read_csv('combined_dataset.csv')
df.head()
```

Out[2]:

| | order_id | customer_id | order_status | order_purchase_timestamp | order_approv |
|---|---|---|---|---|---|
| 0 | e481f51cbdc54678b7cc49136f2d6af7 | 9ef432eb6251297304e76186b10a928d | delivered | 2017-10-02 10:56:33 | 2017-11 |
| 1 | 53cdb2fc8bc7dce0b6741e2150273451 | b0830fb4747a6c6d20dea0b8c802d7ef | delivered | 2018-07-24 20:41:37 | 2018-03 |
| 2 | 47770eb9100c2d0c44946d9cf07ec65d | 41ce2a54c0b03bf3443c3d931a367089 | delivered | 2018-08-08 08:38:49 | 2018-08 |
| 3 | 949d5b44dbf5de918fe9c16f97b45f8a | f88197465ea7920adcdbec7375364d82 | delivered | 2017-11-18 19:28:06 | 2017-19 |
| 4 | ad21c59c0840e6cb83a9ceb5573f8159 | 8ab97904e6daea8866dbdbc4fb7aad2c | delivered | 2018-02-13 21:18:39 | 2018-22 |

In [3]:
```
# Total number of columns in the combined dataset
print(f"Column count: {df.shape[1]}")
```

```
Column count: 18
```

In [4]:
```
# Inspect missing values
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
print(missing[missing > 0].to_frame('missing_count').assign(pct=missing_pct))
```

| | missing_count | pct |
|---|---|---|
| order_approved_at | 160 | 0.16 |
| order_delivered_carrier_date | 1783 | 1.79 |
| order_delivered_customer_date | 2965 | 2.98 |
| review_id | 768 | 0.77 |
| review_score | 768 | 0.77 |
| review_comment_title | 87891 | 88.39 |
| review_comment_message | 58666 | 59.00 |
| review_creation_date | 768 | 0.77 |
| review_answer_timestamp | 768 | 0.77 |

In [5]:
```
# Keep only orders with a confirmed delivery date
df_delivered = df.dropna(subset=['order_delivered_customer_date']).copy()
print(f"Rows retained after filtering undelivered orders: {len(df_delivered)}")
```

```
Rows retained after filtering undelivered orders: 96476
```

In [6]:
```
# Confirm missing values in filtered dataset
df_delivered.isnull().sum()
```

Out[6]:
```
order_id                            0
customer_id                         0
order_status                        0
order_purchase_timestamp            0
order_approved_at                  14
order_delivered_carrier_date        1
order_delivered_customer_date       0
order_estimated_delivery_date       0
review_id                         646
review_score                      646
review_comment_title            85290
review_comment_message          57572
review_creation_date              646
review_answer_timestamp           646
customer_unique_id                  0
customer_zip_code_prefix            0
customer_city                       0
customer_state                      0
dtype: int64
```

In [7]:
```
# Parse delivery date columns as datetime
for col in ['order_estimated_delivery_date', 'order_delivered_customer_date']:
```

```
    df_delivered[col] = pd.to_datetime(df_delivered[col])
```

In [8]:
```
# Compute delivery buffer: positive = delivered early, negative = delivered late
df_delivered['Delivery_Buffer_Days'] = (
    df_delivered['order_estimated_delivery_date'] - df_delivered['order_delivered_customer_date']
).dt.days
```

In [9]:
```
# Quick stats on delivery buffer
print(df_delivered['Delivery_Buffer_Days'].describe().round(2))
```

```
count    96476.00
mean        10.88
std         10.18
min       -189.00
25%          6.00
50%         11.00
75%         16.00
max        146.00
Name: Delivery_Buffer_Days, dtype: float64
```

In [10]:
```
# Classify delivery punctuality
def delivery_category(days):
    if days >= 0:
        return 'On Time'
    elif days >= -5:
        return 'Late'
    else:
        return 'Super Late'

df_delivered['Punctuality'] = df_delivered['Delivery_Buffer_Days'].apply(delivery_category)
```

In [11]:
```
df_delivered.head()
```

Out[11]:

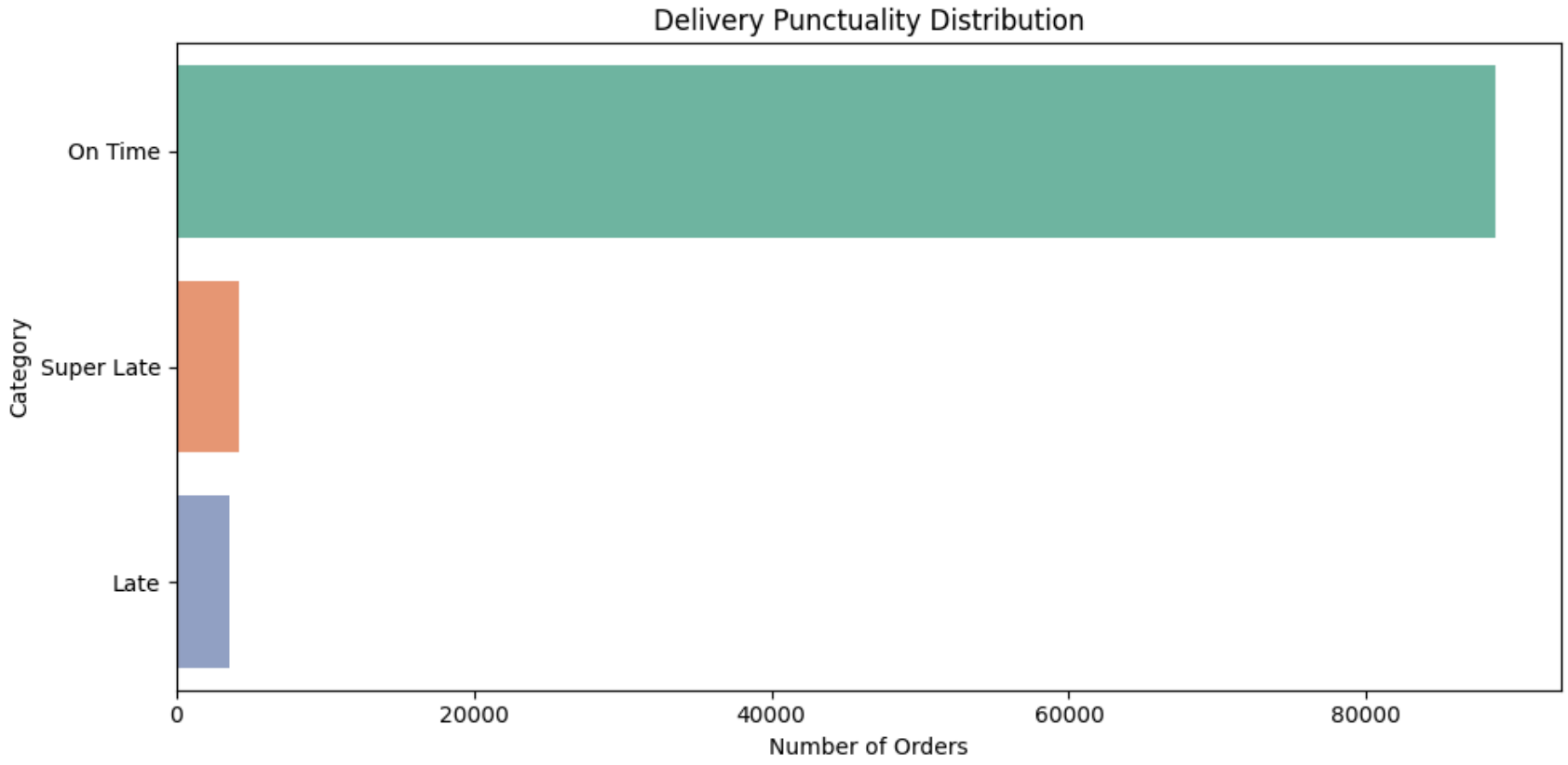
| | order_id | customer_id | order_status | order_purchase_timestamp | order_approv |
|---|---|---|---|---|---|
| 0 | e481f51cbdc54678b7cc49136f2d6af7 | 9ef432eb6251297304e76186b10a928d | delivered | 2017-10-02 10:56:33 | 2017-11 |
| 1 | 53cdb2fc8bc7dce0b6741e2150273451 | b0830fb4747a6c6d20dea0b8c802d7ef | delivered | 2018-07-24 20:41:37 | 2018-03 |
| 2 | 47770eb9100c2d0c44946d9cf07ec65d | 41ce2a54c0b03bf3443c3d931a367089 | delivered | 2018-08-08 08:38:49 | 2018-08 |
| 3 | 949d5b44dbf5de918fe9c16f97b45f8a | f88197465ea7920adcdbec7375364d82 | delivered | 2017-11-18 19:28:06 | 2017-19 |
| 4 | ad21c59c0840e6cb83a9ceb5573f8159 | 8ab97904e6daea8866dbdbc4fb7aad2c | delivered | 2018-02-13 21:18:39 | 2018-22 |

In [12]:
```
# Distribution of delivery punctuality categories
counts = df_delivered['Punctuality'].value_counts()
pcts = df_delivered['Punctuality'].value_counts(normalize=True).mul(100).round(2)
summary = counts.to_frame('count').assign(percentage=pcts)
print(summary)
```

```
             count  percentage
Punctuality
On Time      88649       91.89
Super Late    4212        4.37
Late          3615        3.75
```

In [13]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
order = df_delivered['Punctuality'].value_counts().index
sns.countplot(y=df_delivered['Punctuality'], order=order, palette='Set2')
plt.title('Delivery Punctuality Distribution')
plt.xlabel('Number of Orders')
plt.ylabel('Category')
plt.tight_layout()
plt.show()
```

```
C:\Users\Zaydan\AppData\Local\Temp\ipykernel_12960\2449574176.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(y=df_delivered['Punctuality'], order=order, palette='Set2')
```




In [14]:
```
# Unique customer states in the dataset
states = df_delivered['customer_state'].unique()
print(f"Unique states: {len(states)}")
print(states)
```

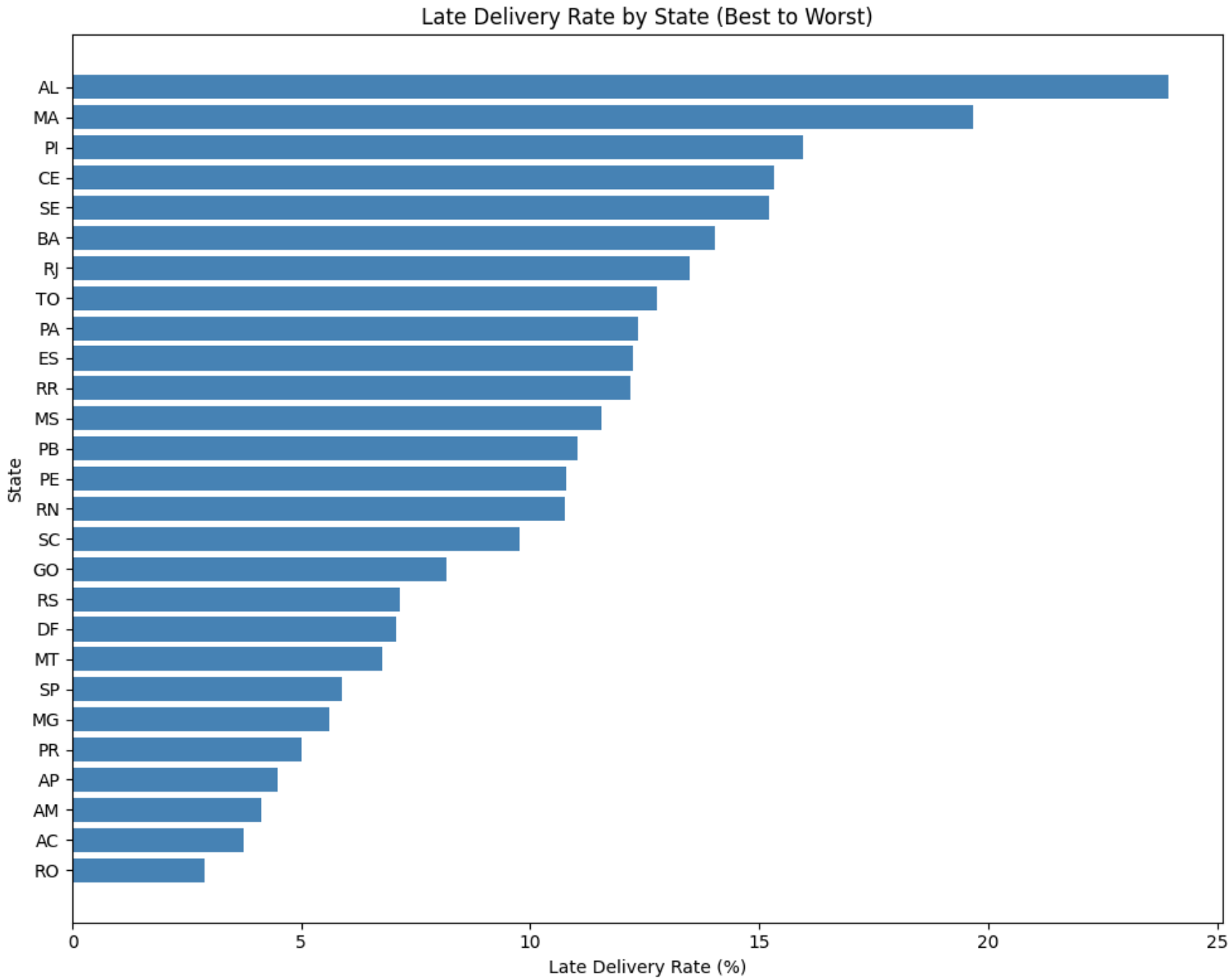
```
Unique states: 27
['SP' 'BA' 'GO' 'RN' 'PR' 'RJ' 'RS' 'MG' 'SC' 'RR' 'PE' 'TO' 'CE' 'DF'
 'SE' 'MT' 'PB' 'PA' 'RO' 'ES' 'AP' 'MS' 'MA' 'PI' 'AL' 'AC' 'AM']
```

In [15]:
```
# Flag whether order arrived after estimated delivery date
df_delivered['delivered_late'] = (
    df_delivered['order_delivered_customer_date'] > df_delivered['order_estimated_delivery_date']
)
```

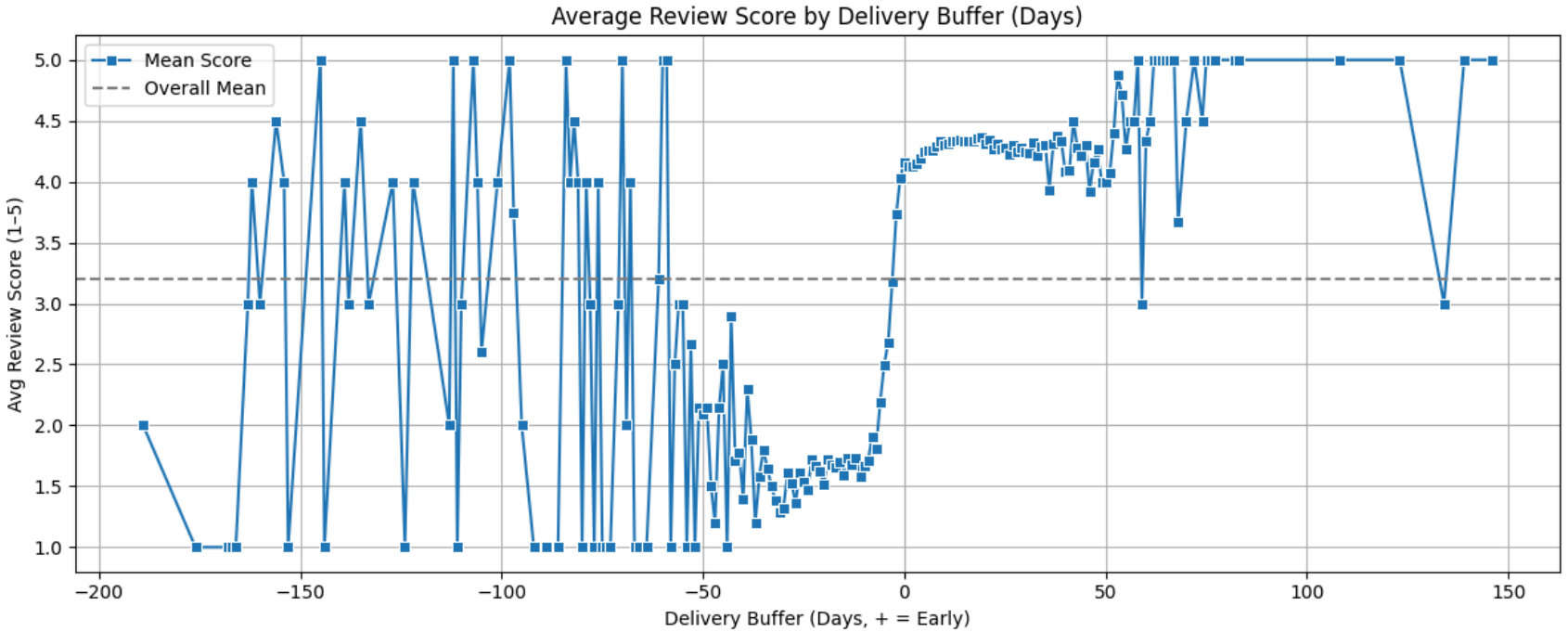
In [16]:
```
# Aggregate late delivery rate by state
state_stats = df_delivered.groupby('customer_state').agg(
    total_orders=('order_id', 'count'),
    late_count=('delivered_late', 'sum')
).reset_index()

state_stats['late_rate'] = (state_stats['late_count'] / state_stats['total_orders']) * 100
```

In [17]:
```
# Sort by late rate — best performers at top
state_ranked = state_stats.sort_values(by='late_rate', ascending=True)
```

In [18]:
```
plt.figure(figsize=(10, 8))
plt.barh(state_ranked['customer_state'], state_ranked['late_rate'], color='steelblue')
plt.title('Late Delivery Rate by State (Best to Worst)')
plt.xlabel('Late Delivery Rate (%)')
plt.ylabel('State')
plt.tight_layout()
plt.show()
```


Late Delivery Rate by State (Best to Worst)
State
AL
MA
PI
CE
SE
BA
RJ
TO
PA
ES
RR
MS
PB
PE
RN
SC
GO
RS
DF
MT
SP
MG
PR
AP
AM
AC
RO
0
5
10
15
20
25
Late Delivery Rate (%)

In [20]:
```
# Average review score grouped by delivery buffer (rounded to nearest day)
df_delivered['Buffer_Day_Bucket'] = df_delivered['Delivery_Buffer_Days'].round()
score_by_buffer = (
    df_delivered
    .groupby('Buffer_Day_Bucket')['review_score']
    .agg(['mean', 'median', 'count'])
    .reset_index()
)

plt.figure(figsize=(12, 5))
sns.lineplot(data=score_by_buffer, x='Buffer_Day_Bucket', y='mean', marker='s', label='Mean Score')
plt.axhline(y=score_by_buffer['mean'].mean(), color='gray', linestyle='--', label='Overall Mean')
plt.title('Average Review Score by Delivery Buffer (Days)')
plt.xlabel('Delivery Buffer (Days, + = Early)')
plt.ylabel('Avg Review Score (1-5)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [21]:
```
# Average review score by punctuality category
punctuality_review = (
    df_delivered
    .groupby('Punctuality')['review_score']
    .agg(['mean', 'std', 'count'])
    .round(2)
)
print(punctuality_review)
```

```
             mean   std  count
Punctuality
Late         3.46  1.56   3568
On Time      4.29  1.15  88168
Super Late   1.78  1.31   4094
```

In [22]:
```
# Load order items and product details
items_df = pd.read_csv('olist_order_items_dataset.csv')
products_df = pd.read_csv('olist_products_dataset.csv')
items_with_products = items_df.merge(products_df, how='left', on='product_id')
```

In [23]:
```
# Attach product info to delivered orders
df_delivered = df_delivered.merge(items_with_products, how='left', on='order_id')
```

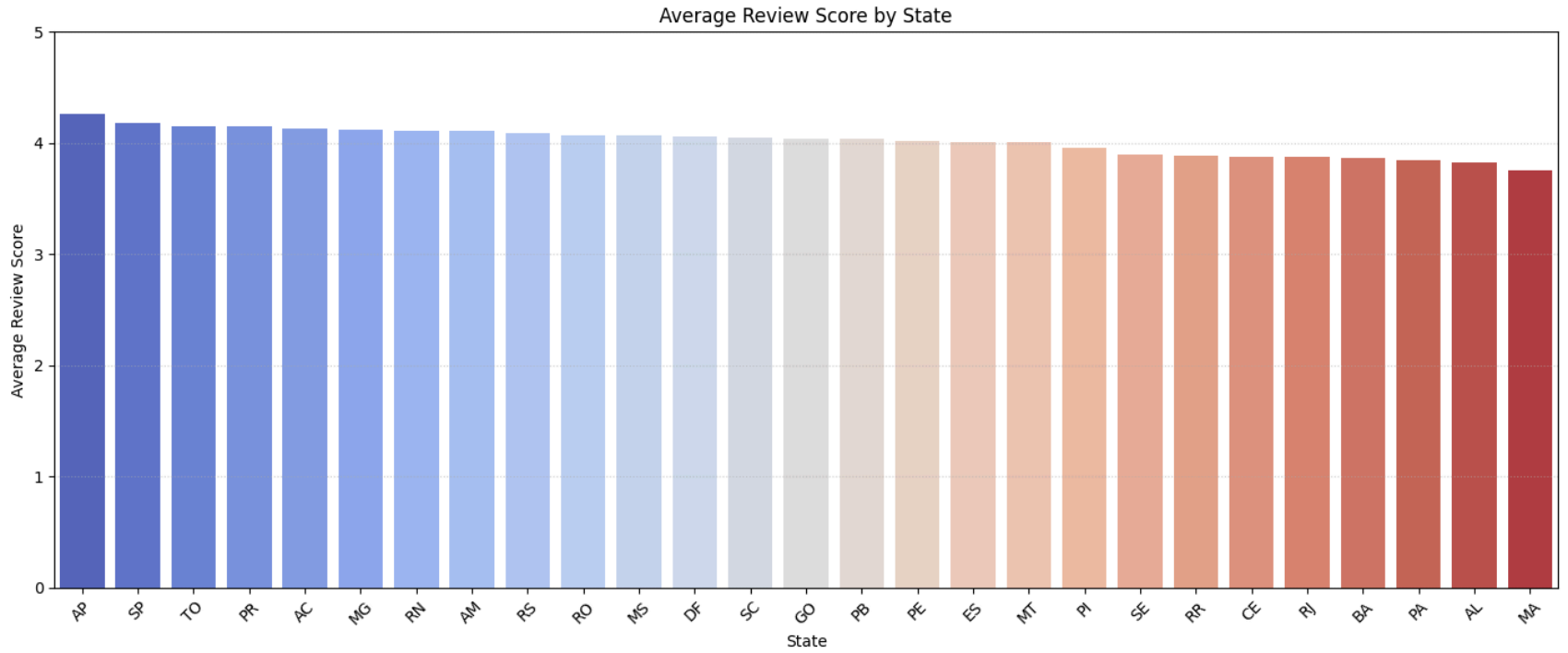
In [24]:
```
# Load and attach English product category names
cat_trans = pd.read_csv('product_category_name_translation.csv')
df_delivered = df_delivered.merge(cat_trans, how='left', on='product_category_name')

# Verify
df_delivered[['product_category_name', 'product_category_name_english']].head()
```

Out[24]:

| | product_category_name | product_category_name_english |
|---|---|---|
| **0** | utilidades_domesticas | housewares |
| **1** | perfumaria | perfumery |
| **2** | automotivo | auto |
| **3** | pet_shop | pet_shop |
| **4** | papelaria | stationery |

In [25]:
```
# Compute average review score per state
state_review = df_delivered.groupby('customer_state').agg(
    avg_review=('review_score', 'mean')
).reset_index()

# Join with state_stats for fuller picture
state_analysis = state_stats.merge(state_review, on='customer_state')
state_analysis = state_analysis.sort_values('avg_review', ascending=False)

plt.figure(figsize=(14, 6))
sns.barplot(data=state_analysis, x='customer_state', y='avg_review', palette='coolwarm')
plt.title('Average Review Score by State')
plt.xlabel('State')
plt.ylabel('Average Review Score')
plt.xticks(rotation=45)
plt.ylim(0, 5)
plt.grid(axis='y', linestyle=':', alpha=0.6)
plt.tight_layout()
plt.show()
```

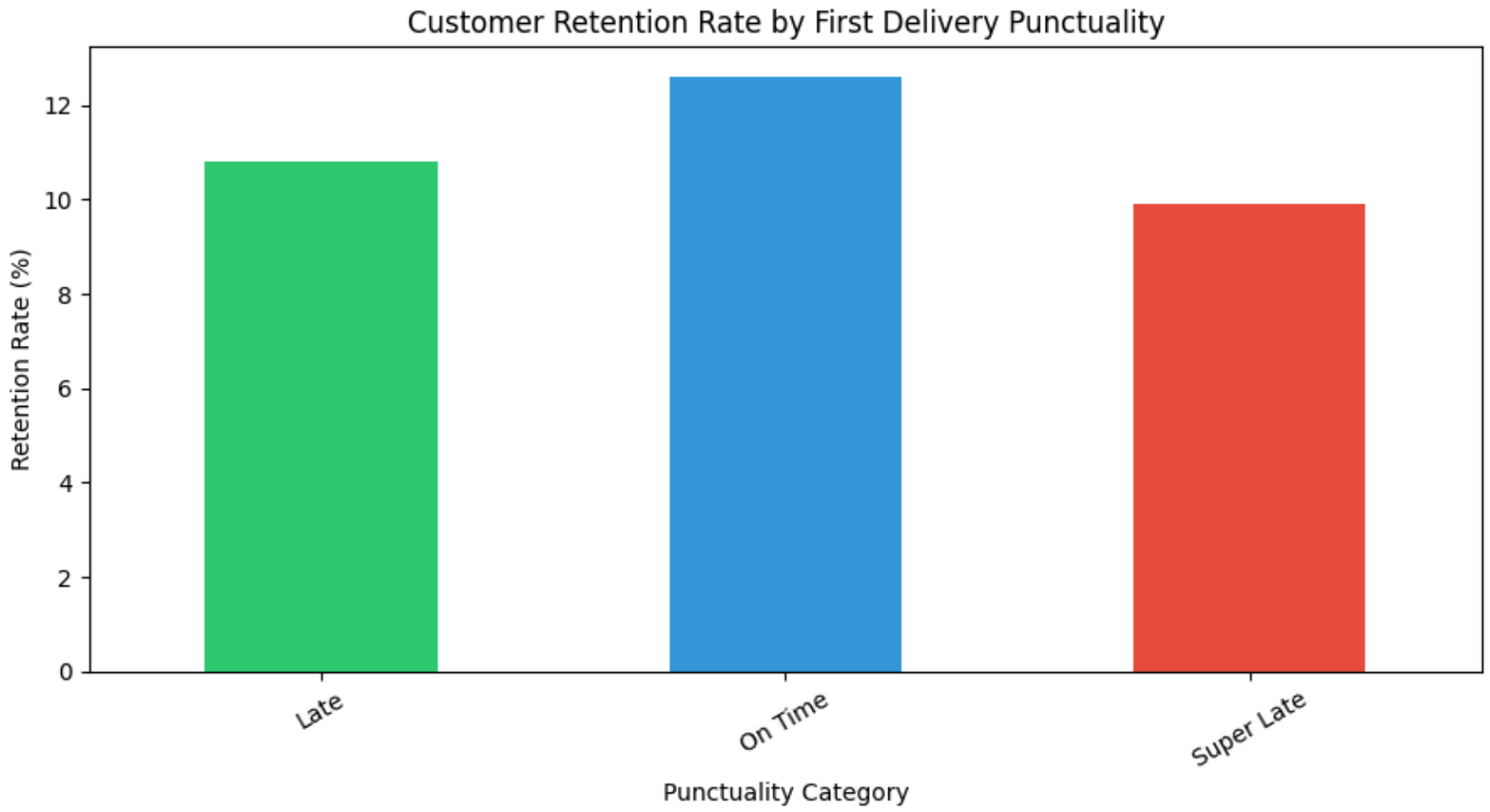
```
C:\Users\Zaydan\AppData\Local\Temp\ipykernel_12960\2929502594.py:11: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `h
ue` and set `legend=False` for the same effect.

  sns.barplot(data=state_analysis, x='customer_state', y='avg_review', palette='coolwarm')
```




In [26]:
```python
# Identify first-time vs repeat customers and compute retention by punctuality
df_delivered = df_delivered.sort_values(by=['customer_unique_id', 'order_purchase_timestamp'])
df_delivered['visit_number'] = df_delivered.groupby('customer_unique_id').cumcount() + 1

# Flag customers who made more than one purchase
repeat_customers = df_delivered[df_delivered['visit_number'] > 1]['customer_unique_id'].unique()

# Focus on first visits
first_visits = df_delivered[df_delivered['visit_number'] == 1].copy()
first_visits['came_back'] = first_visits['customer_unique_id'].isin(repeat_customers)

retention = first_visits.groupby('Punctuality').agg(
    new_customers=('customer_unique_id', 'count'),
    returned=('came_back', 'sum')
)
retention['retention_pct'] = (retention['returned'] / retention['new_customers'] * 100).round(1)
print(retention)

retention['retention_pct'].plot(
```

```
    kind='bar',
    color=['#2ecc71', '#3498db', '#e74c3c', '#f39c12'],
    figsize=(9, 5),
    rot=30
)
plt.title('Customer Retention Rate by First Delivery Punctuality')
plt.ylabel('Retention Rate (%)')
plt.xlabel('Punctuality Category')
plt.tight_layout()
plt.show()
```

| Punctuality | new_customers | returned | retention_pct |
|---|---|---|---|
| Late | 3512 | 378 | 10.8 |
| On Time | 85751 | 10829 | 12.6 |
| Super Late | 4093 | 404 | 9.9 |


Customer Retention Rate by First Delivery Punctuality
Retention Rate (%)
0
2
4
6
8
10
12
Late
On Time
Super Late
Punctuality Category

In [ ]: